Operating System Project

Linux System Debugging Tool

Team Members :
2520030012 - Dharmika
2520030501 - Anushka K
2520030542 - Rushvini

Introduction

The **Linux System Debugging Tool** is designed to monitor and analyze Linux processes. It helps users understand how a program interacts with the Linux operating system by tracing its system calls, reading CPU registers, examining memory, and monitoring system resources. The tool uses Linux system-level debugging techniques to collect this information and presents the results through a simple and user-friendly web dashboard.

Problem Statement

Debugging processes in Linux can be complex because understanding a program's internal behavior requires examining different aspects such as **system calls, CPU registers, memory usage, and system resources**. Traditional debugging methods often involve multiple commands and tools, making the process difficult for beginners to understand. It can also be challenging to get a clear view of how a process interacts with the operating system during execution.

Solution

Our **Linux System Debugging Tool** provides a unified and user-friendly way to monitor and analyze a Linux process. It observes the process during execution and collects important information such as **system calls, CPU registers, memory usage, and system resource consumption**. The debugging data is processed by the system and presented in a **web-based dashboard**, where users can easily view and understand the process activity. This helps students and beginners learn how Linux processes work internally and understand important **Operating System concepts** in a practical way.

Objectives



Objective 1

To monitor and trace Linux processes by observing their system calls and execution behavior during runtime.

Objective 2

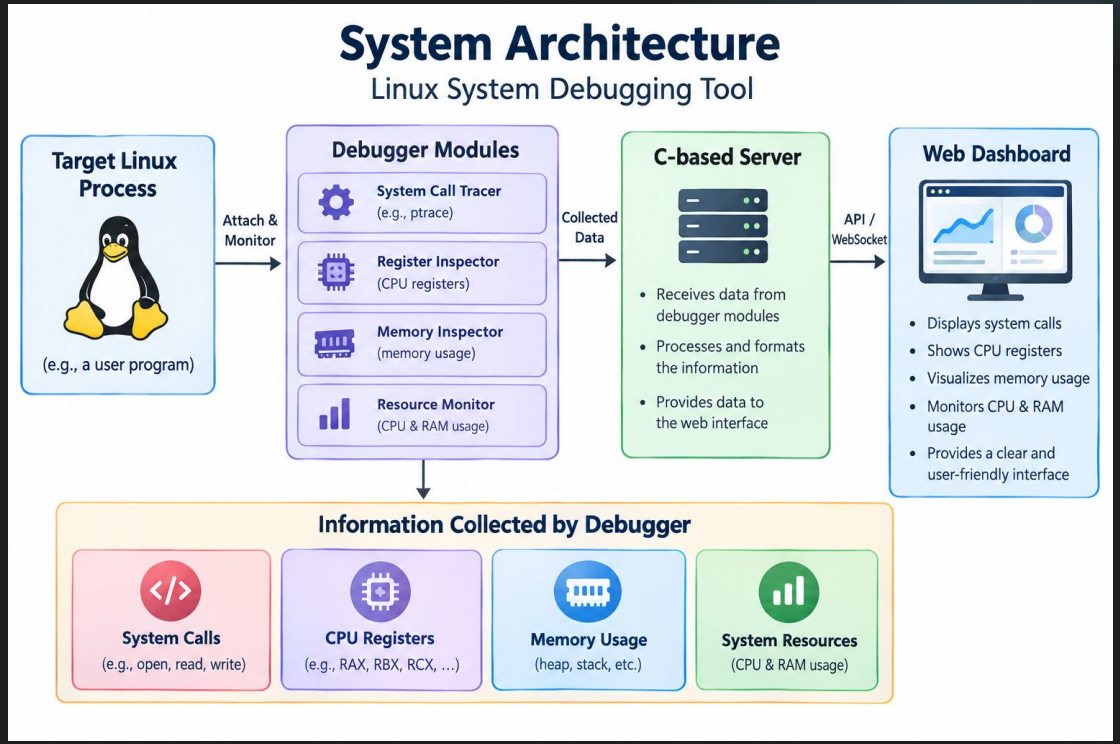
To analyze system-level information such as CPU registers, memory usage, and CPU/RAM resource consumption.

Objective 3

To display debugging information clearly through a simple web-based dashboard for easier understanding and analysis.

Proposed System

Our system consists of a **target Linux process, debugging modules, a C-based server, and a web dashboard**. The debugger monitors the target process and collects information related to **system calls, CPU registers, memory, and system resources**. The C server connects the debugger with the web interface, and the collected results are displayed on the dashboard in a clear and organized manner.



Technologies Used

- **C Programming** – Used to develop the core debugging modules.
- **Linux / Ubuntu** – Provides the operating system environment and system-level features.
- **ptrace()** – Used for process control and system-call tracing.
- **process_vm_readv()** – Used to access process memory.
- **getrusage() & sysinfo()** – Used to monitor CPU and memory resources.
- **HTML, CSS & JavaScript** – Used to create the web-based dashboard.
- **C-based HTTP Server** – Connects the debugger with the web interface

Implementation / Modules

- **Process Control:** Starts the target process, creates the child process, and manages its execution using Linux process-control mechanisms.
- **System Call Tracer:** Monitors the system calls made by the target process and records their execution during runtime.
- **Register Reader:** Reads important CPU register values to understand the processor state while the process is being executed.
- **Memory Reader:** Examines the memory used by the target process to understand its memory-related activities.
- **Resource Monitor:** Collects information about CPU time, RAM usage, and other system resources used during execution.
- **Web Server & Dashboard:** Connects the debugging modules with the web interface and presents the collected information in a clear and organized format.

Outputs

Our project successfully demonstrates the monitoring and analysis of a Linux target process. The debugger collects and displays different types of system-level information during execution.

- **Process Information:** Displays the target process and its execution status.
- **System Calls:** Shows the system calls performed by the target process.
- **CPU Registers:** Displays register information collected during process execution.
- **Memory Information:** Provides details related to the process memory.
- **Resource Usage:** Shows CPU time, total RAM, free RAM, and available RAM.
- **Web Dashboard:** Presents all the collected debugging information in a simple and organized interface.

Future Scope

- **Multiple Process Debugging:** The tool can be extended to monitor and analyze multiple Linux processes at the same time, making it more useful for larger applications.
- **Advanced Memory Analysis:** Future versions can provide detailed information about memory allocation, memory leaks, and different memory regions used by a process.
- **Interactive Debugging:** Features such as **pause, resume, step-by-step execution, and process control** can be added for more detailed debugging.
- **Improved Data Visualization:** The dashboard can include **graphs and charts** to show CPU usage, memory usage, and system-call activity more clearly.
- **Advanced Monitoring Features:** Additional Linux performance metrics and debugging features can be integrated to make the tool more comprehensive and useful for learning and development.

Conclusion

Our **Linux System Debugging Tool** applies important **Operating System concepts such as process management, system calls, memory management, CPU and register management, and resource management**. In real-time, it can be used by developers to monitor a running Linux process and identify issues such as **high CPU or memory usage, unexpected system calls, and abnormal process behavior**. The web dashboard makes this information easy to understand and helps in **debugging, performance analysis, and learning how Linux processes work internally**.

Thank you!